

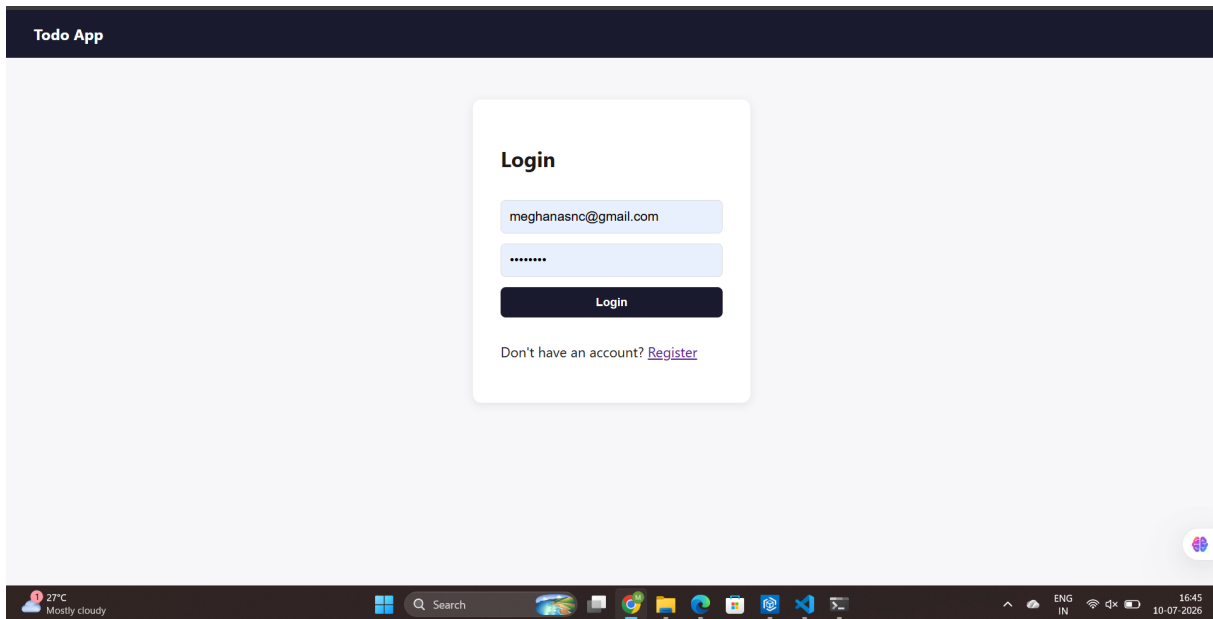
Project Screenshots

AI-Powered Task & Knowledge Management System — output screenshots covering authentication, RBAC, document upload, task management, AI search, and API documentation.

1. Authentication & RBAC

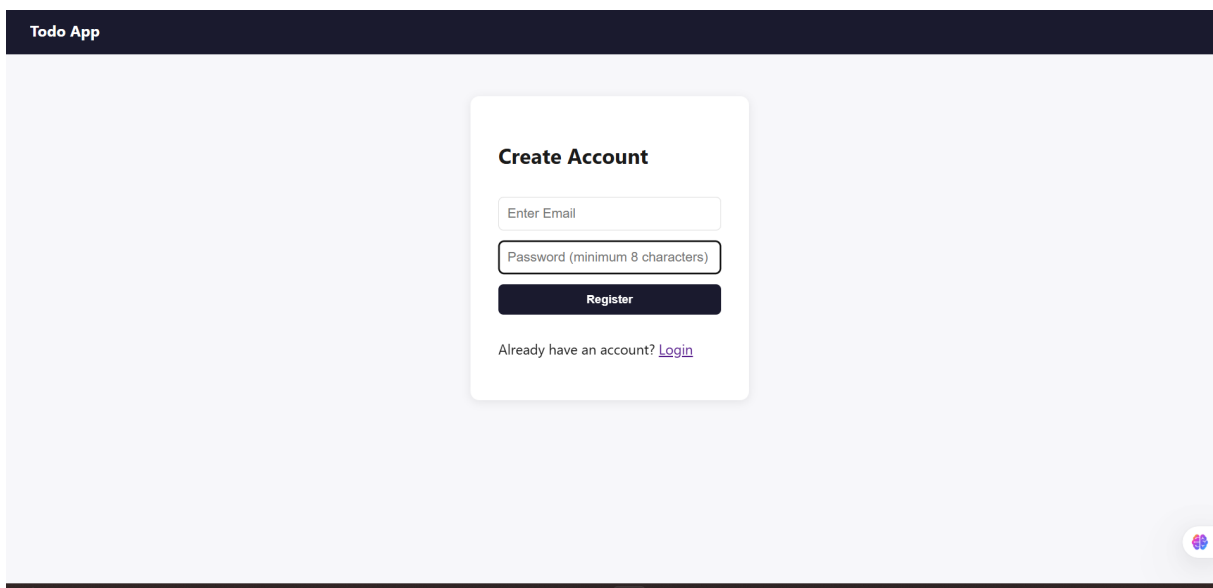
Login Page

01_login_page.png



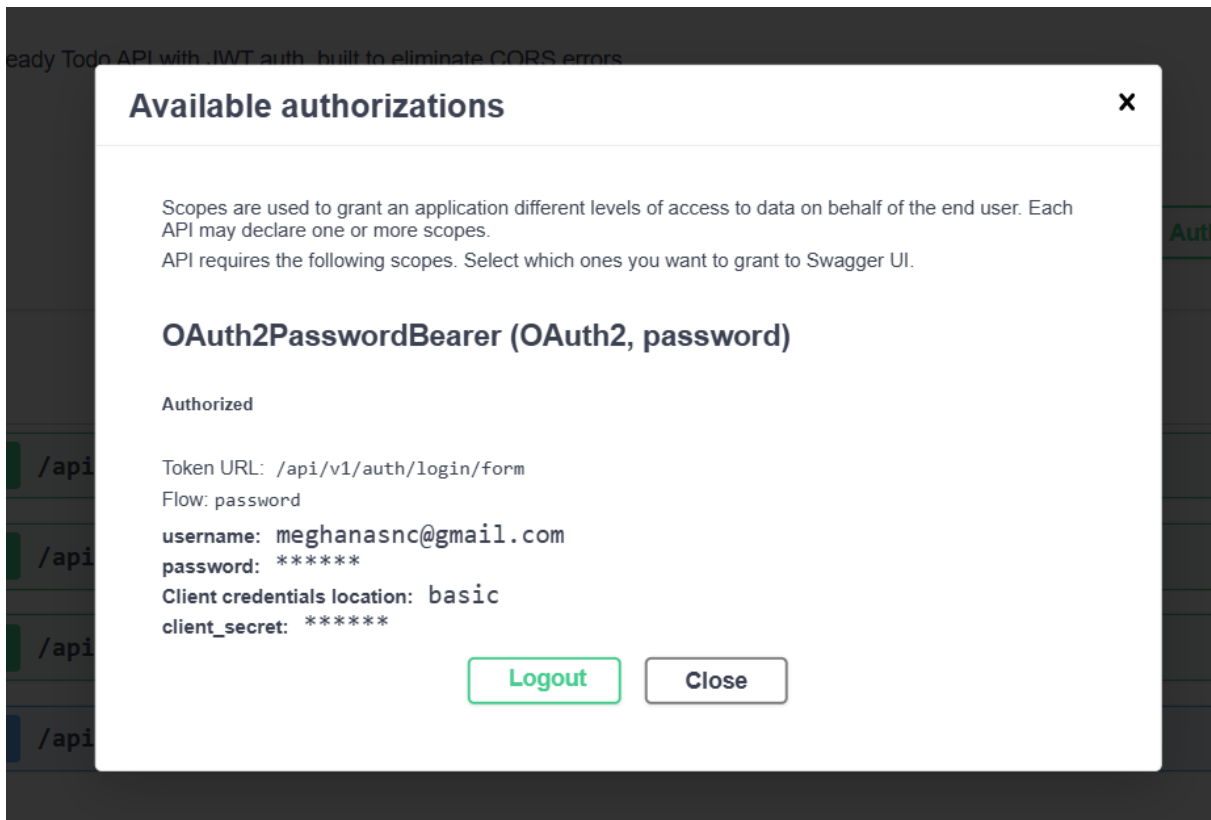
Register Page

02_register_page.png



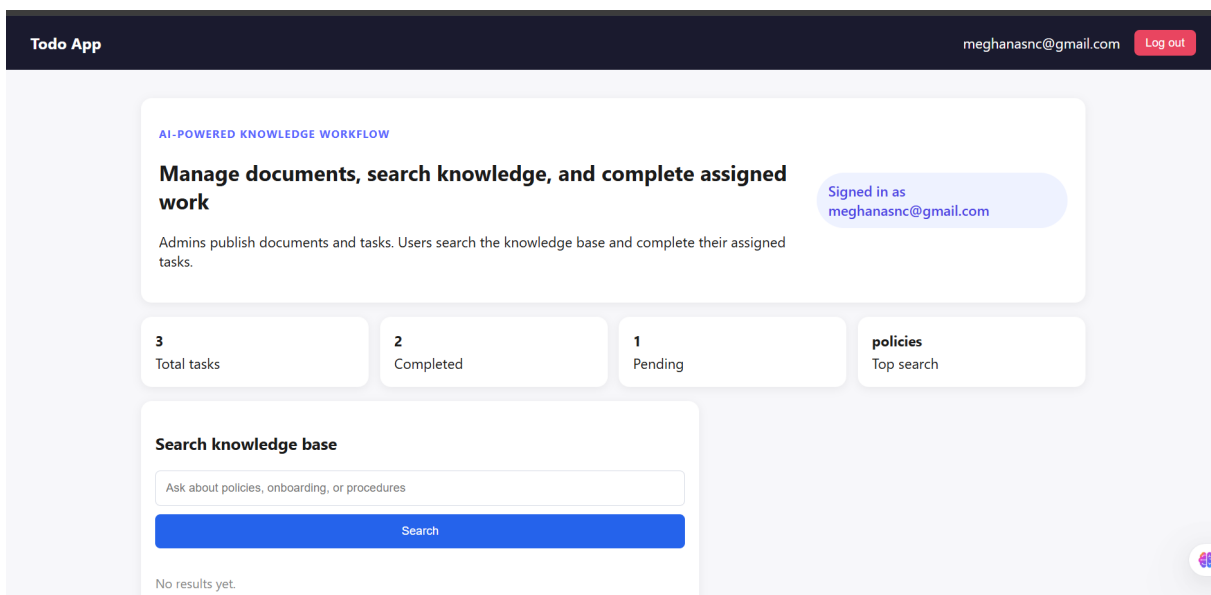
JWT Authorization (Swagger)

03_jwt_authorize_swagger.png



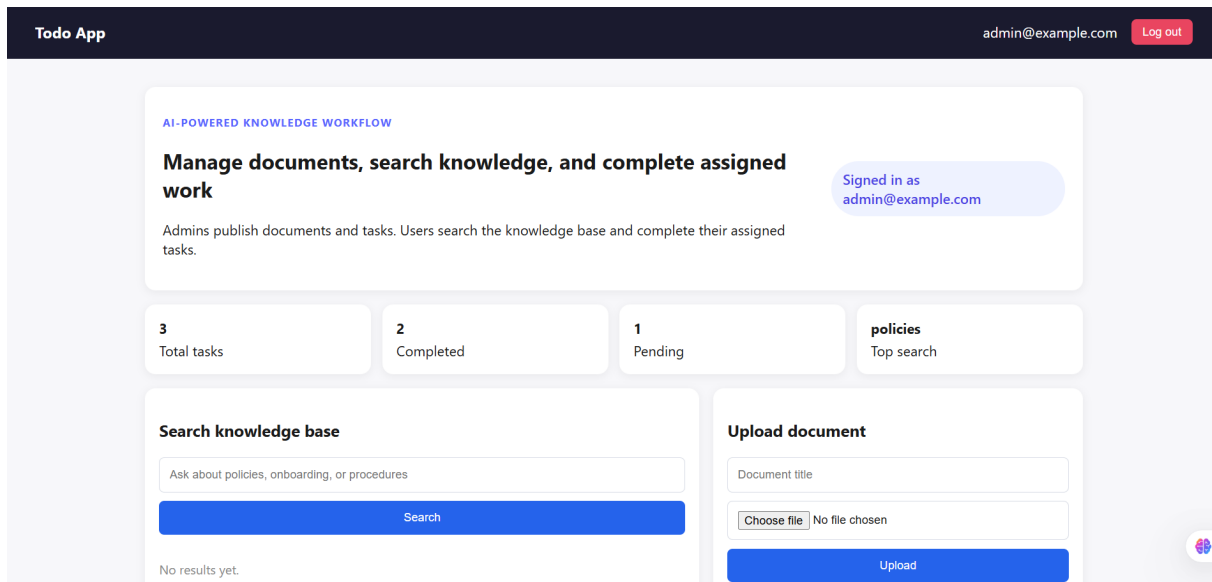
User Dashboard (Role: User)

04_user_dashboard.png



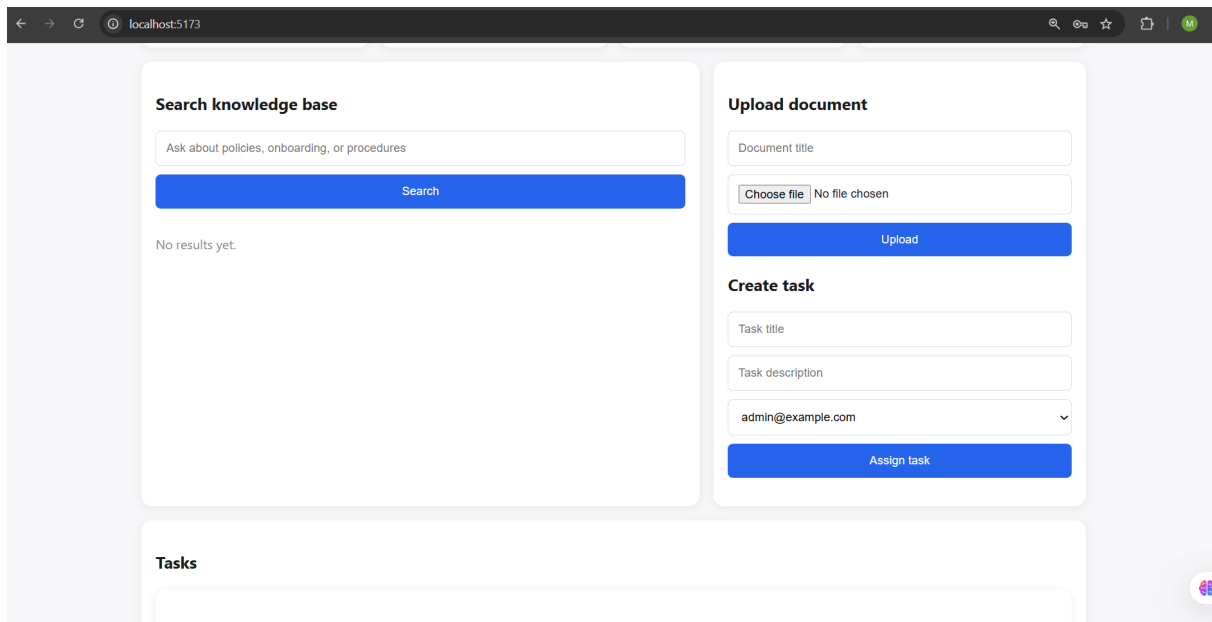
Admin Dashboard (Role: Admin)

05_admin_dashboard.png



Admin Full Controls — Upload, Create Task, Assign Task

06_admin_full_controls.png



2. Document Upload

Upload Document Form

07_upload_document_form.png

The screenshot shows a web application interface for document management. At the top, a dark header bar contains the text 'Todo App' on the left, 'admin@example.com' in the center, and a 'Log out' button on the right. Below the header, a main content area features a white card with the title 'AI-POWERED KNOWLEDGE WORKFLOW' and a subtitle 'Manage documents, search knowledge, and complete assigned work'. A description below reads: 'Admins publish documents and tasks. Users search the knowledge base and complete their assigned tasks.' To the right of the card, a blue bubble indicates 'Signed in as admin@example.com'. Below the card, four white boxes display statistics: '3 Total tasks', '2 Completed', '1 Pending', and 'policies Top search'. The main content area is divided into two columns. The left column, titled 'Search knowledge base', contains a search input field with the placeholder text 'Ask about policies, onboarding, or procedures', a blue 'Search' button, and the text 'No results yet.' below it. The right column, titled 'Upload document', contains a 'Document title' input field, a 'Choose file' button next to the text 'No file chosen', and a blue 'Upload' button at the bottom. A small circular icon with a brain symbol is visible in the bottom right corner of the main content area.

3. Task Management

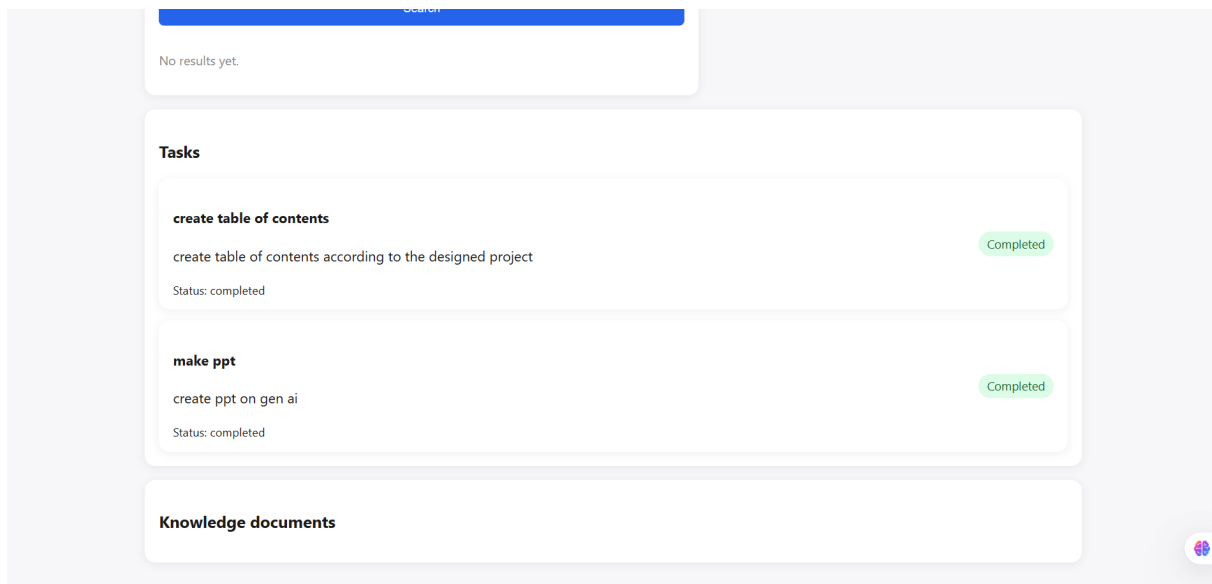
Task List with Completion Status

08_tasks_list_completed.png

The screenshot shows a web application interface for task management. At the top, a dark header bar contains the text 'Todo App' on the left, 'admin@example.com' in the center, and a 'Log out' button on the right. Below the header, a main content area features a white card with the title 'Tasks'. The card contains three task entries, each with a title, description, status, and a 'Completed' button. The first task is 'Complete Project Documentation' with the description 'Prepare API documentation and update project README file' and status 'Status: completed'. The second task is 'create table of contents' with the description 'create table of contents according to the designed project' and status 'Status: completed'. The third task is 'make ppt' with the description 'create ppt on gen ai' and status 'Status: completed'. Below the task list, a section titled 'Knowledge documents' is visible. A small circular icon with a brain symbol is visible in the bottom right corner of the main content area.

Tasks + Knowledge Documents Section

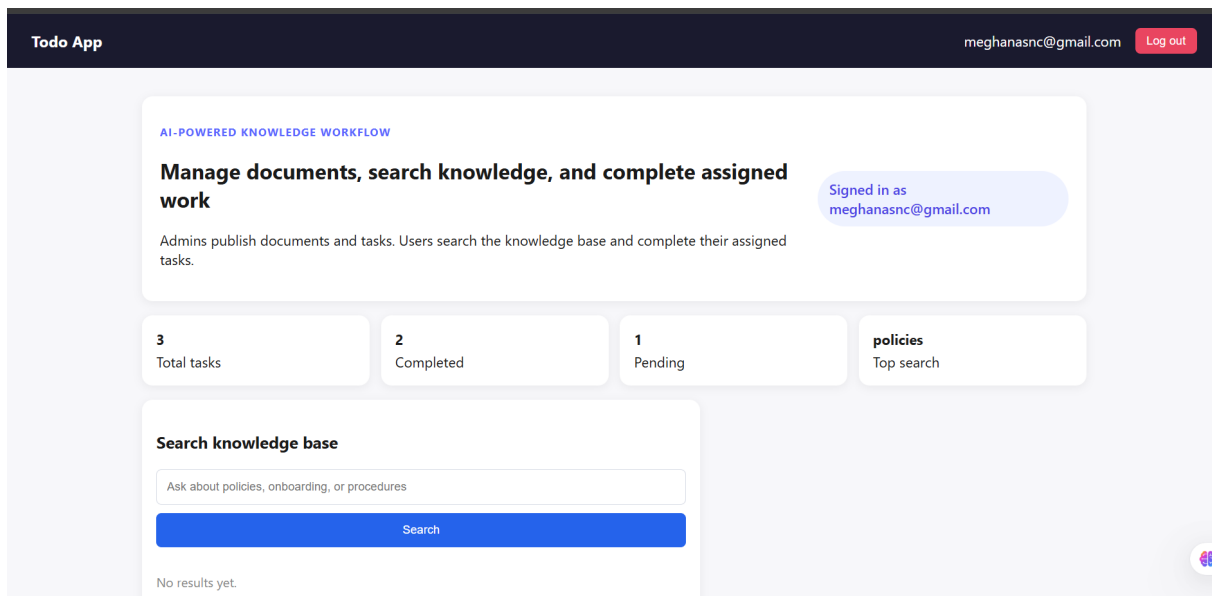
09_tasks_and_knowledge_section.png



4. AI-Powered Search

Search Knowledge Base UI

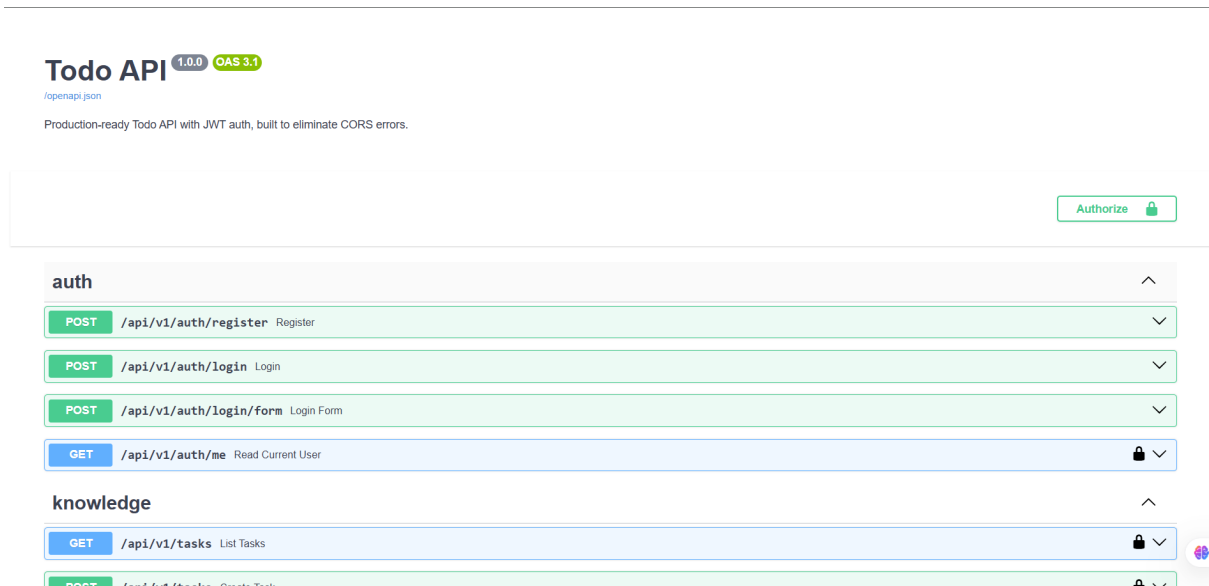
10_search_ui_empty.png



5. API Documentation (Swagger)

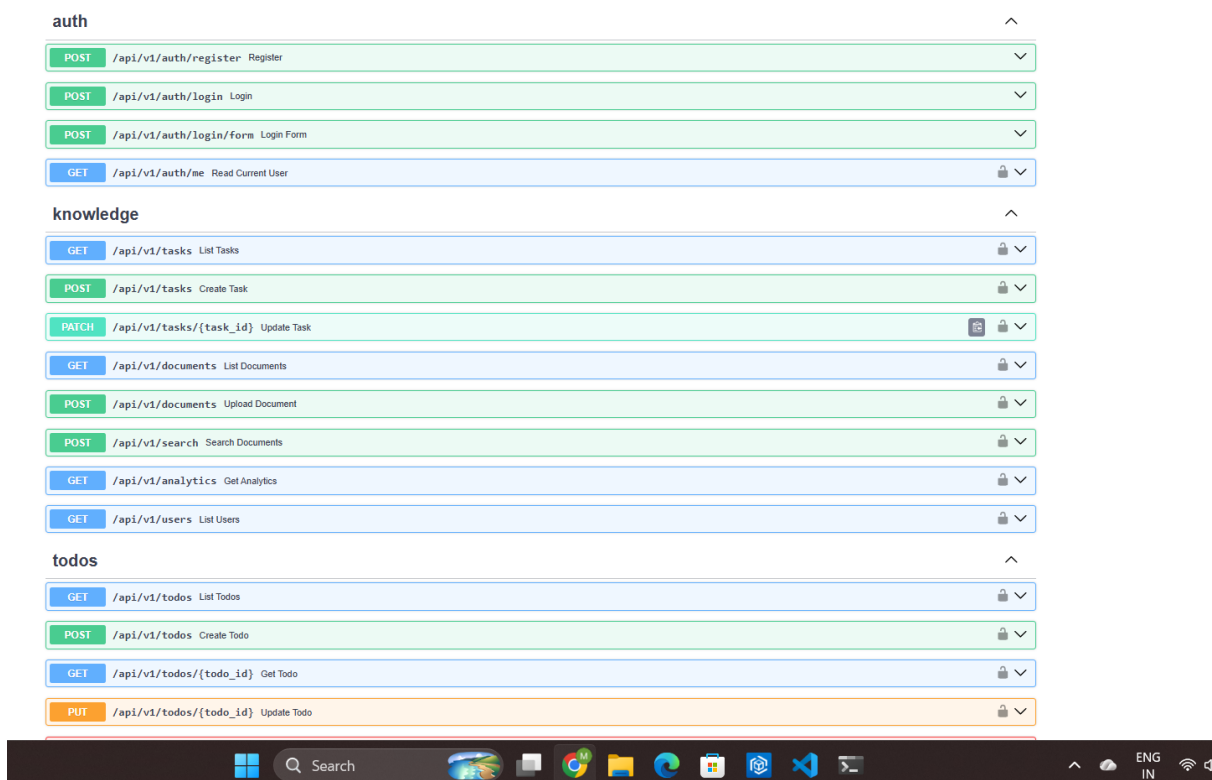
Swagger Overview — Todo API

11_swagger_overview.png



All Endpoints (auth, knowledge, todos)

12_swagger_all_endpoints.png



GET /todos/{id} — Parameters

13_get_todo_params.png

The screenshot shows a REST client interface with a dark theme. At the top, a JSON snippet is visible: `{ "owner_id": 0 }`. Below it, the endpoint `POST /api/v1/todos` is selected, with the description "Create Todo". The next endpoint, `GET /api/v1/todos/{todo_id}`, is highlighted in blue, with the description "Get Todo". Under the "Parameters" tab, a table lists the parameter `todo_id` as a required integer (path). A text input field next to it contains the value `4`. Below the table are "Execute" and "Clear" buttons. The "Responses" section is empty. The "Curl" section displays a generated curl command: `curl -X 'GET' \ 'http://127.0.0.1:8000/api/v1/todos/4' \ -H 'accept: application/json' \ -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3ZWR1I0I3tZWdoYW5hc25jQ6dtYH1sLnNvbSIzImV4cCI6MTc4MzY4NDMxNm0.kprubZBbX-iHgbFU1P4F8F5a03P4m2pE16T2N7WvZhw'`. The "Request URL" section shows `http://127.0.0.1:8000/api/v1/todos/4`.

GET /todos/{id} — Generated Curl Command

14_get_todo_curl.png

This screenshot is similar to the previous one but includes additional UI elements. The "Parameters" table and "Execute/Clear" buttons are the same. In the "Responses" section, there is a "Cancel" button. The "Curl" section shows the same generated curl command as before. The "Request URL" section shows `http://127.0.0.1:8000/api/v1/todos/4`. The interface also features a lock icon and a dropdown arrow in the top right corner of the endpoint selection area.

GET /todos/{id} — 404 Response Example

15_get_todo_404_response.png

The screenshot shows a REST client interface with the URL `http://127.0.0.1:8000/api/v1/todos/4`. The 'Server response' section displays a 404 status code with the message 'Error: Not Found'. The response body is a JSON object: `{ "detail": "todo with id 4 was not found." }`. The response headers are: `content-length: 42`, `content-type: application/json`, `date: Fri, 18 Jul 2020 10:54:54 GMT`, and `server: uvicorn`. Below this, the 'Responses' section shows a table with two entries: a 200 'Successful Response' and a 422 'Validation Error'. The 200 response is expanded, showing a media type of 'application/json' and an example JSON object: `{ "title": "string", "description": "", "id": 0, "completed": true, "owner_id": 0 }`.

PUT/DELETE Todos + Health Check

16_put_delete_health_endpoints.png

The screenshot shows a REST client interface with a JSON object in the top editor: `{ "loc": [ "string", 0 ], "msg": "string", "type": "string" }`. Below the editor, there are three endpoint cards: a PUT endpoint `/api/v1/todos/{todo_id}` for 'Update Todo', a DELETE endpoint `/api/v1/todos/{todo_id}` for 'Delete Todo', and a GET endpoint `/api/v1/health` for 'Health Check'. The 'health' endpoint is expanded, showing a description: 'Simple endpoint to verify the API is reachable — use this first when debugging CORS.' It also shows a 'Parameters' section with 'No parameters' and a 'Responses' section with a 200 'Successful Response'.

Data Schemas (TaskOut, TodoOut, DocumentOut, etc.)

17_swagger_schemas.png

Schemas			^
AnalyticsOut >	Expand all	object	
Body_login_form_api_v1_auth_login_form_post >	Expand all	object	
Body_upload_document_api_v1_documents_post >	Expand all	object	
DocumentOut >	Expand all	object	
HTTPValidationError >	Expand all	object	
TaskCreate >	Expand all	object	
TaskOut >	Expand all	object	
TaskUpdate >	Expand all	object	
TodoCreate >	Expand all	object	
TodoOut >	Expand all	object	
TodoUpdate >	Expand all	object	
Token >	Expand all	object	
UserCreate >	Expand all	object	
UserOut >	Expand all	object	
ValidationError >	Expand all	object	

Documents & Search Endpoints

18_documents_search_endpoints.png

PATCH	/api/v1/tasks/{task_id}	Update Task	🔒	⌵
GET	/api/v1/documents	List Documents	🔒	⌵
POST	/api/v1/documents	Upload Document	🔒	⌵
POST	/api/v1/search	Search Documents	🔒	⌵
GET	/api/v1/analytics	Get Analytics	🔒	⌵
GET	/api/v1/users	List Users	🔒	⌵
todos				^
GET	/api/v1/todos	List Todos	🔒	⌵
Parameters				
No parameters				
Execute Clear				
Responses				

19_get_todos_empty_response.png

